

Operating Systems (OS)

Inter Process Communication (IPC) III

Process Cooperation

Prof. Dr.-Ing. Jörg Abke



TH Aschaffenburg
university of applied sciences

Copyright Hinweise

© Jörg Abke

Dieses Werk ist urheberrechtlich geschützt. Alle Rechte, insbesondere Vervielfältigung, Verbreitung oder Übersetzung liegen beim Autor. Kein Teil des Werkes darf ohne Genehmigung des Autors, auch nicht für Unterrichtszwecke, fotokopiert oder durch andere Verfahren reproduziert oder verbreitet werden. Ebenfalls ist die Einspeicherung in EDV-Anlagen ohne Genehmigung des Autors nicht gestattet. Jeder Studierende, der an der entsprechenden Lehrveranstaltung des Autors an der Technischen Hochschule Aschaffenburg teilnimmt, ist berechtigt, einen Ausdruck dieses Werkes anzufertigen und für die Zwecke seines Studiums zu verwenden. Eine weitergehende Verwendung bedarf in jedem Falle der ausdrücklichen Zustimmung des Autors. Gebrauchsname, Handelsnamen, Warenbezeichnungen etc. werden in diesem Werk ohne Gewährleistung der freien Verwendbarkeit benutzt. Texte, Abbildungen, Programme und technische Angaben wurden sorgfältig erarbeitet. Trotzdem sind Fehler nicht völlig auszuschließen. Der Autor weist darauf hin, dass er für Fehlerfreiheit keine Gewähr und für eventuelle Folgen keine Haftung übernimmt.

Inter-Process Communication Intro

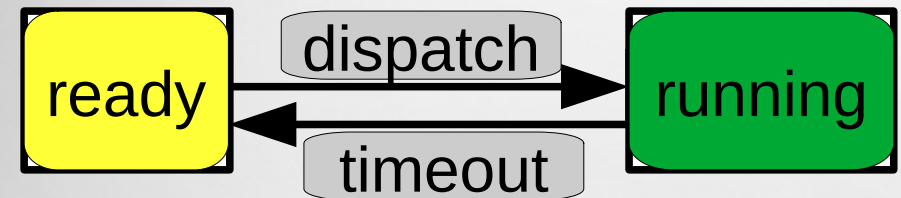
Processes

- Undertake read, write operations on data
- Call each other (e.g. forking)
- Wait for each other

Interaction by processes – Inter-Process Communication (IPC)

→ following questions:

- How to transmit/exchange data
- How to access shared resources



Similar questions for threads

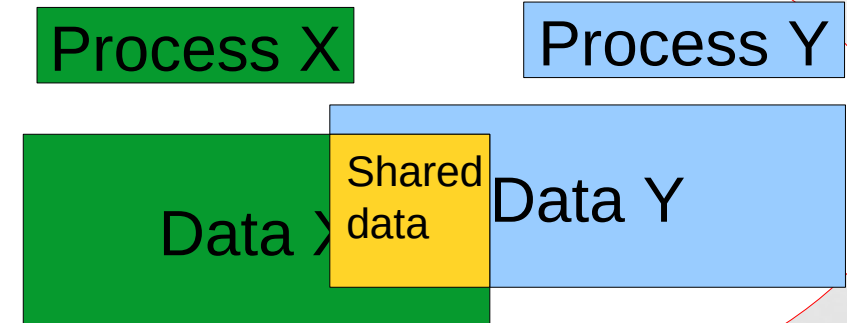
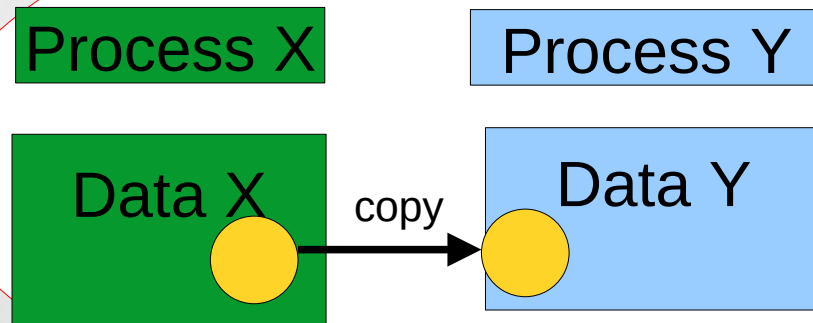
Communication, Cooperation



TH Aschaffenburg
university of applied sciences

Communication

Cooperation



This chapter

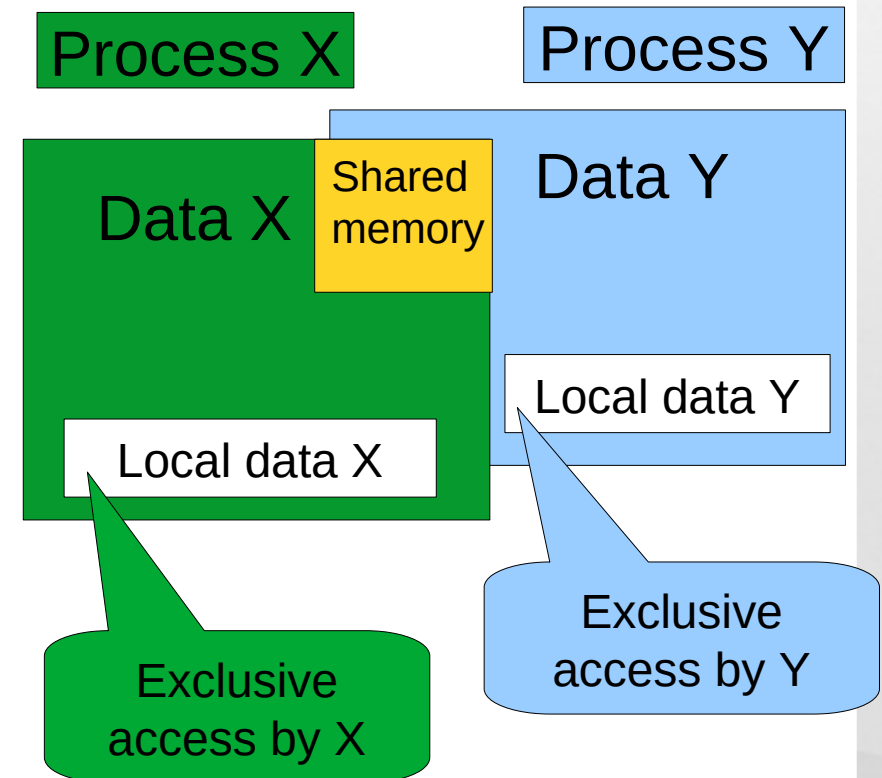
Synchronisation

- **Processes to share data and resources**
- **Challenges**
 - Data consistency due to concurrency
 - Race conditions
 - (Dead-) Locking
- Realization by
 - Unix System Vor
 - **Posix System**

Shared Memory (ShMem)

Purpose:

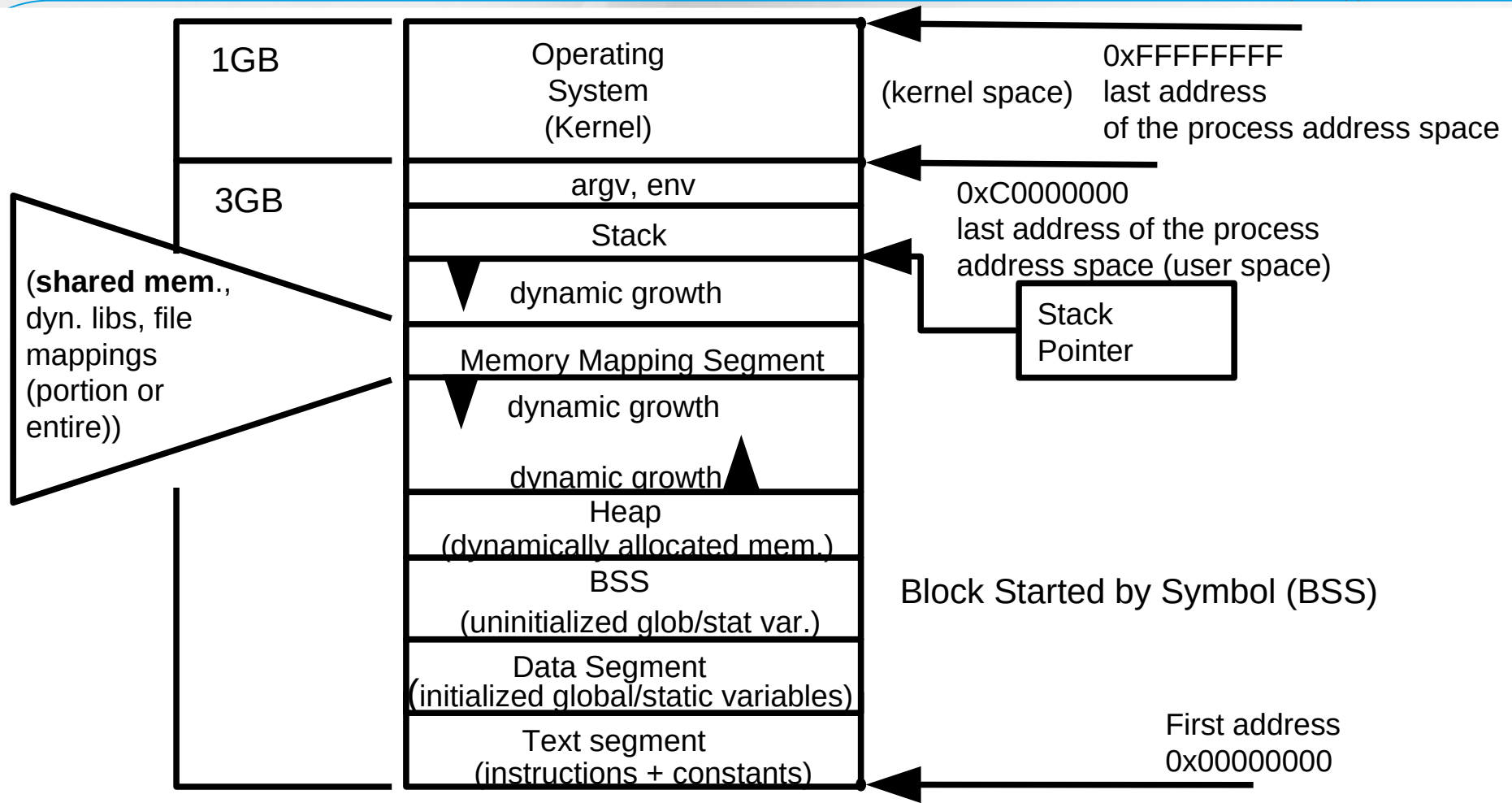
- For memory based communication and data exchange
- Located in address space of multiple processes (→ next slide)
- All processes need to coordinate data access to ShMem, due to
 - Race conditions
 - Data inconsistencies



Process Structure in Memory (e.g. 32 bit Linux, with 4GB mem *sandbox*)



TH Aschaffenburg
university of applied sciences



Source: C. Baun, Operating Systems – bilingual

See also: <https://unix.stackexchange.com/questions/466443/do-memory-mapping-segment-and-heap-grow-until-they-meet-each-other>

Shared Memory in Unix

- UNIX operating systems contain a shared memory table, which contains information about the existing shared memory segments
 - Start address in memory
 - size
 - Owner (username and group)
 - Privileges
- Content of a ShMem is **not erased by end/termination of processes**, which are attached to it
- Reboot of a system leads to data loss in ShMem

System Calls with Posix ShMem

- **shm_open()**: Create a shared memory segment or access an existing one
- **ftruncate()**: Specify the size of a shared memory segment
- **mmap()**: Attach a shared memory segment to a process
- **munmap()**: Detach a shared memory segment from a process
- **close()**: Close the descriptor of a shared memory segment
- **shm_unlink()**: Erase a segment

Note: Formal parameters are left in this overview!

- POSIX shared memory segments in Linux can be found in the directory
/dev/shm

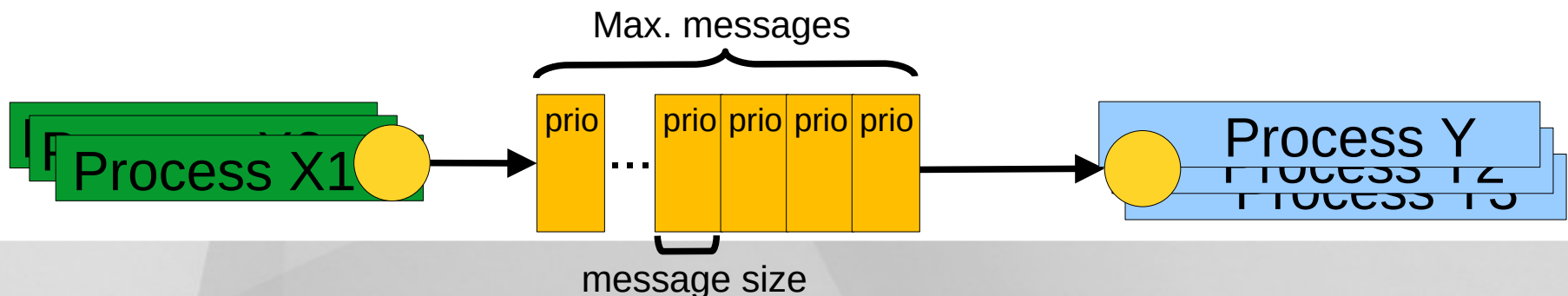
Example ShMem Writer and Reader in C with Posix



- **9_3_Shared_Mem_Writer_Posix**
- **9_3_Shared_Mem_Reader_Posix**
- **Let's have a look after one run:**
 - `$> ls -la /dev/shm/`
drwxrwxrwt 3 root root 180 Nov 25 13:29 .
drwxr-xr-x 21 root root 7200 Nov 25 09:42 ..
-rw-rw-r-- 1 abke abke 1024 Nov 25 13:26 MySharedMem
 - `$> more /dev/shm/MySharedMem`
Hello from POSIX writer!

Message Queue (mq)

- Linked list with messages
- **F**irst In **F**irst **O**ut principle
- **A process stores or some processes store messages** in a message queue, which can be fetched from there by
 - the (sending) process
 - or
 - other receiving process or processes
- Messages may contain priorities



C-System Calls for Message Queues in Posix



TH Aschaffenburg
university of applied sciences

- **mq_open():** Create a message queue or access an existing one
- **mq_send():** Write (send) a message into a message queue. Blocking operation
- **mq_timedsend():** Write (send) a message into a message queue. Blocking operation with a timeout
- **mq_receive():** Read (receive) a message from a message queue. Blocking operation
- **mq_timedreceive():** Read (receive) a message from a message queue. Blocking operation with a timeout
- **mq_getattr():** Request the attributes of a message queue. These are: number of messages in the queue, maximum message size, maximum number of messages. . .
- **mq_setattr():** Modify the attributes of a message queue
- **mq_notify():** Notify the process as soon as a message is available
- **mq_close():** Close a message queue
- **mq_unlink():** Erase a message queue

Note: Formal parameters are left in this overview!

Example mqueue Writer and Reader in C with Posix



- **9_3_mQueue_Posix_sender**
- **9_3_mQueue_Posix_receiver**
- **\$ ls -la /dev/mqueue/**
drwxrwxrwt 2 root root 140 Nov 26 17:08 .
drwxr-xr-x 21 root root 7200 Nov 26 15:28 ..
-rw-r--r-- 1 abke abke 80 Nov 26 17:08 MyMsgQueue

Pipe - Pipeline

- **2 Categories**
 - Anonymous pipe
 - Named pipe

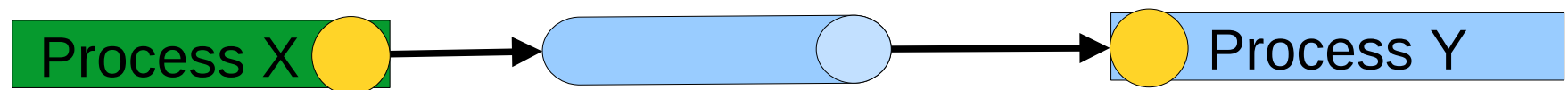
Example from shell with | as pipe

\$> ls -la | more

- „ls -la output on **standard output is piped** to *more*, which **reads on standard input stream**“
- ls and more are respective processes connected by pipe

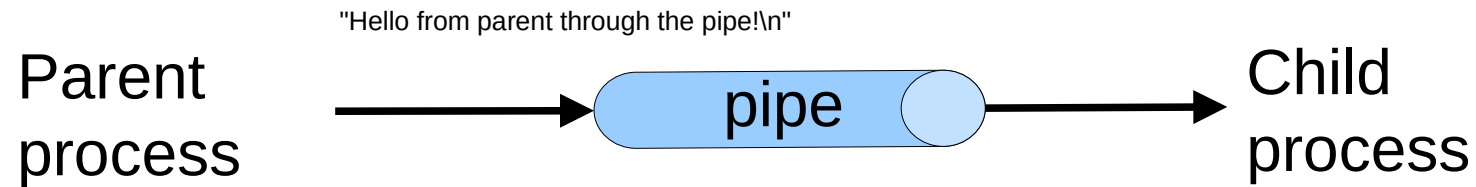
Anonymous Pipe

- Buffered unidirectional communication channel between 2 processes
- Buffer size is
- **FIFO** principle
- Limited capacity
(for Linux: *ulimit -a* reports pipe size (512 bytes, -p) 8, $8 \cdot 512 \text{b} = 4 \text{kb}$)
 - Pipe filled → writing is blocked
 - Pipe empty → reading is blocked
- OS creates anonymous pipe and delete it as soon as one process ends
 - **pipe()**: creates an anonymous pipe
 - **write()**: sends data to pipe
 - **read()**: reads data from pipe



Example for an Anonymous Pipe

- **9_3_anonymous_pipe.c**



Named Pipe - FIFO

- Every process can write to and read from, which know the name
- At least 2 processes participate (1 sender, 1 receiver)
- **mkfifo()**: creates a named pipe with access rights
- **open()**: opens a named pipe
- **read()**: reads from a named pipe
- **write()**: writes to a named pipe
- **close()**: closes a named pipe
- **unlink()**: deletes a named pipe

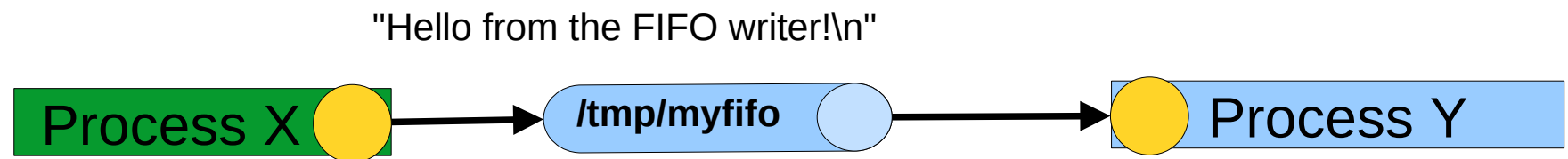
Note: Formal parameters are left in this overview!

POSIX named pipes in Linux can be found as a file

Example for Named Pipe

- 9_3_named_pipe_writer
- 9_3_named_pipe_reader (to be started at first)

both use `fifo(-file) /tmp/myfifo`



Comparison of Inter Process Communication Means



TH Aschaffenburg
university of applied sciences

	Shared Memory	Message Queue	Anonymous Pipe	Named Pipe	Socket
Memory vs Message based com.	Memory	Message	Message	Message	Message
Direction	Bidirectional	Bidirectional	Unidirectional	Unidirectional	Bidirectional
Process relation to each other	no	no	yes	no	no
Over system boundaries	no	no	no	no	yes
Persists even without bound process(es)	yes	yes	no	yes	no
Access Synchronisation	no	yes	yes	yes	yes

Inter Process Communication

- **Shared Memory ✓**
- **Message Queue ✓**
- **Anonymous Pipe ✓**
- **Named Pipe ✓**
- **Sockets (→ Courses: Verteilte und Parallele Systeme / Distributed and Parallel Systems)**